LIMS Print Pipeline

Complete reference — architecture, data flows, security model, and a step-by-step guide to testing it from a fresh machine

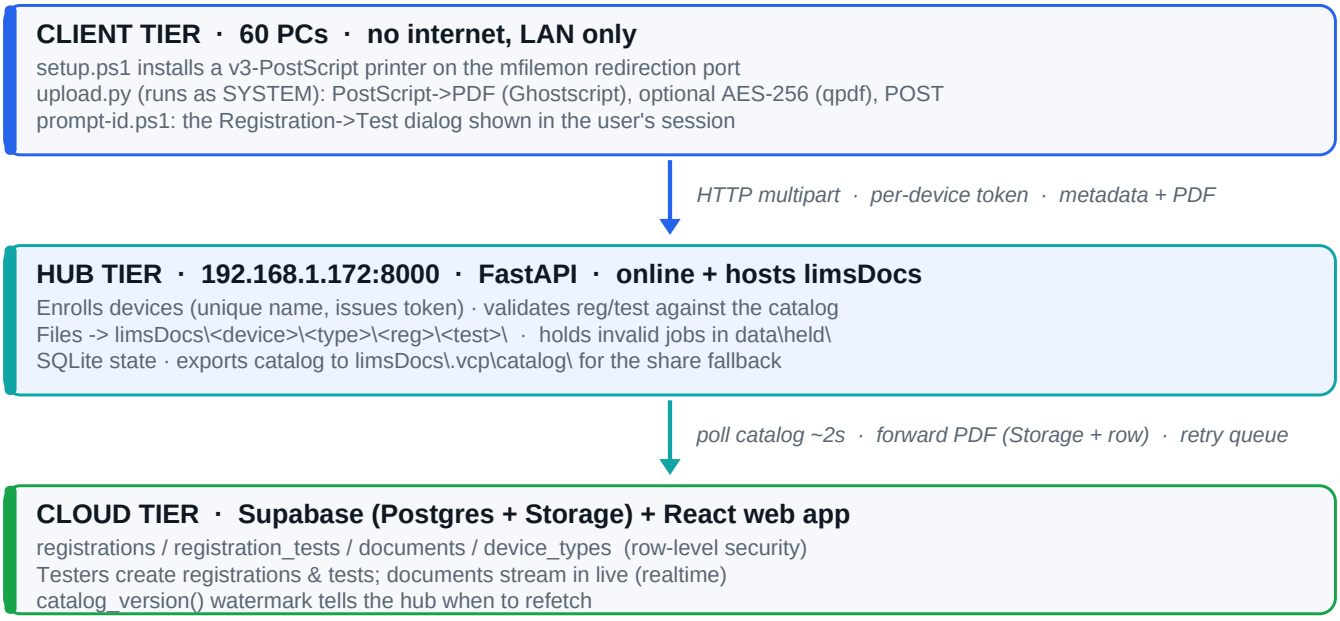
1 What it is

A Windows **virtual-printer** toolkit turned into a laboratory document pipeline. Staff on 60 internet-isolated PCs print from any application; each job becomes a PDF, is tagged with a **registration number** and **test** chosen from live dropdowns, is filed into a shared folder on a central desktop, and is forwarded to a cloud **LIMS** (Supabase + React). Registration numbers originate in the LIMS, so nothing is typed by hand; a job without a valid one is safely **held** until an operator assigns it. It runs fully on the LAN — only the one central desktop needs internet.

2 System at a glance

Component	Runs on	Responsibility
Client printer	each of 60 PCs (offline)	Capture print -> PDF; prompt Registration->Test; POST to hub
Central hub	central desktop 192.168.1.172 (online)	File PDFs into limsDocs tree; hold unregistered; forward to cloud
Cloud LIMS	Supabase + React (internet)	Testers create registrations; documents appear live
Registration catalog	cloud -> hub -> clients	Syncs every ~2s; drives the client dropdowns
limsDocs share	central desktop	Canonical filed-document tree, reachable by all clients

3 Architecture



4 Data flows

4.1 A print job (the happy path)

1 · Print
User prints from any app -> v3 driver emits PostScript -> mfilemon writes it to spool\

↓

2 · Prompt
upload.py GETs the catalog for this device type; the dialog shows Registration then Test dropdowns

↓

3 · Convert & send
Ghostscript makes the PDF (optionally AES-256); POST to /ingest/<token> with reg, test, sha256, device…

↓

4 · File
Hub validates against the catalog -> saves to limsDocs\<device>\<type>\<reg>\<test>\<stamp>_<doc>.pdf

↓

5 · Forward
Upload to Supabase Storage + insert a documents row -> appears in the LIMS within seconds

4.2 The safety path (nothing is lost)

A registration is **required**. Three ways a job can miss one, and what happens:

Situation	Result
User clicks Skip / dialog times out	Job uploads anyway -> hub marks it HELD (missing_registration)
Number/test doesn't match the catalog	HELD with the exact reason (unknown_registration, unknown_test, device_type_mismatch, missing_test)
Hub briefly unreachable	upload.py keeps the PDF in failed\ and retries; never dropped
Cloud (Supabase) unreachable	Hub keeps the file, marks forward_failed, retries with backoff until it succeeds

Held documents wait in the hub dashboard's **Held** tab. An operator picks the correct registration + test -> the file moves into the limsDocs tree and forwards to the cloud.

4.3 Catalog sync (cloud -> clients)

Tester creates a registration in the LIMS
reg_no + device type + tests -> registrations / registration_tests

↓ *catalog_version() watermark changes*

Hub notices within ~2s and refetches
Replaces its cached catalog in one transaction; exports .vcp\catalog\<type>.json to the share

↓ *client GET /catalog (or share file if offline)*

Client dialog shows it as a dropdown option
Filtered to the printer's device type; searchable; test list follows the chosen registration

5 Security model

The pipeline moves confidential lab documents across 60 machines and the internet, so security is layered at every hop.

- **Per-device tokens.** Each printer carries its own secret ingest token in its URL. The hub's device record — not any client-sent field — is authoritative for the device's name and type. Revoking a device kills only its token.
- **Enroll key & admin token.** Registering a device needs the enroll key; the dashboard and all admin APIs need the admin token. Both are random, generated on the hub's first start, compared in constant time. Only /healthz and the dashboard HTML are open.
- **Passwords encrypted at rest.** Per-printer PDF passwords are stored as machine-bound DPAPI blobs (LocalMachine scope, fixed entropy) — never plaintext. Viewing one needs the super-admin password, itself stored only as a PBKDF2-SHA256 hash (200k iterations).
- **Cloud key stays on-site.** The Supabase service-role key lives only on the central desktop, DPAPI-encrypted. The web app uses the limited anon key plus row-level security; testers can read but never insert documents (only the hub does, server-side).
- **Path-traversal proof.** Every folder segment built from client input (device, type, registration, test, document name) passes one sanitizer. A malicious document name like ..\..\evil cannot escape the limsDocs tree — verified in tests.
- **Encryption fails closed.** If encryption is enabled but the password or qpdf is missing, the job is preserved in failed\ rather than sent in the clear. A wrong DPAPI key raises rather than silently returning garbage.
- **Locked install folder.** %ProgramData%\VirtualCloudPrinter is restricted to SYSTEM + Administrators. The print user is granted only the minimum (create its own spool file; read+execute the interpreter) — closing the local privilege-escalation path.
- **TLS-ready transport.** LAN traffic can run over HTTPS (verify_tls per printer); the cloud legs are HTTPS. Confidential production data should always front the hub with TLS.

6 Key contracts (for maintainers)

Contract	Value
Filing tree	limsDocs__.pdf
Ingest fields	docname, registration_number, test, device_name, device_type, user, job_id, sha256, encrypted, file
Hub -> cloud	POST Storage object (x-upsert:false, 409=ok) then POST /rest/v1/documents; retry 5s->300s
DPAPI entropy	UTF-8 'VCP-DPAPI-v1', LocalMachine — identical in setup.ps1, upload.py, hub/app.py
Super-admin	PBKDF2-SHA256, 200k iters, 16B salt (factory: Citta@26252423 — rotate on day one)

Full contract detail lives in **ARCHITECTURE.md**; deployment in **LAN-SETUP.md**.

7 Testing it from fresh

Two tracks. **Track A** proves the entire pipeline on **one Windows PC** in ~15 minutes (no LAN, no Supabase needed) — do this first. **Track B** is the real multi-machine deployment.

Before you start (Track A prerequisites)

Windows 10/11 x64 · Administrator rights (installers self-elevate via UAC) · internet on first run (downloads Ghostscript, mfilemon, Python once) · a copy of this project folder.

Track A — single-PC smoke test

1. **Start the hub** In a terminal: `cd hub` then `run.bat`. Leave Supabase unset — it runs in **local mode**.
   ✓ Console prints an ADMIN TOKEN and an ENROLL KEY. Dashboard opens at http://localhost:8000/

2. **Open the dashboard** Browse to http://localhost:8000/, paste the ADMIN TOKEN, and go to **Settings**. Set the limsDocs directory to a local folder, e.g. C:\limsDocs (create it). Save.
   ✓ Settings show 'local mode'; C:\limsDocs\.vcp\catalog is created.

3. **Add a registration** On the **Catalog** tab, add one: reg no e.g. R-TEST-1, device type gcms, tests rx-test, ry-test. (This stands in for what a tester would create in the cloud LIMS.)
   ✓ R-TEST-1 appears in the catalog table with both tests.

4. **Install a client printer** Double-click `install.bat` (approve UAC). Enter: printer name GCMS Printer · hub URL http://localhost:8000 · device name gcms-01 · the ENROLL KEY · device type gcms · PDF password (optional) · accept the catalog-share default.
   ✓ Finishes with 'Printer GCMS Printer is ready'; hub Devices tab lists gcms-01.

5. **Print a test document** Open anything (Notepad, a PDF, Word) -> Print -> choose **GCMS Printer**.
   ✓ A dialog pops up showing the Registration dropdown (R-TEST-1) and, after you pick it, the Test dropdown.

6. **Pick & attach** Choose R-TEST-1, then rx-test, then **Attach & print**.
   ✓ The dialog closes and the job proceeds.

7. **Verify filed** Check C:\limsDocs\gcms-01\gcms\R-TEST-1\rx-test\ — and the hub **Documents** tab.
   ✓ A .pdf is in that folder; the document shows status 'filed' in the dashboard.

8. **Verify the safety path** Print again but click **Skip** on the dialog.
   *✓ The hub **Held** tab shows the job (reason missing_registration). Assign R-TEST-1 + a test -> it moves into the tree.*

If the dialog doesn't appear or nothing files

Run `fix-queue.bat` (elevated) — it repairs permissions, restarts the spooler and writes vcp-diagnostics.txt. Check %ProgramData%\VirtualCloudPrinter\log.txt for the per-job trace, and the hub console for what it received. Unposted PDFs are always kept in failed\.

Track B — full LAN + cloud deployment

Once Track A works, scale out. Full detail is in **LAN-SETUP.md**; the shape is:

1. **Cloud** Create a Supabase project -> run lims/supabase/schema.sql in the SQL editor -> enable email auth + add tester logins -> start the React app (lims/web) with the project URL + anon key.
2. **Hub** On the central desktop (192.168.1.172): run.bat -> in Settings enter the Supabase URL + service-role key and the limsDocs local path. Share the limsDocs folder to the LAN; open firewall port 8000.
3. **Clients** On each of the 60 PCs: install.bat with the hub URL, a UNIQUE device name, its device type, a PDF password, and the enroll key. (Scripted one-liner in LAN-SETUP.md for mass rollout.)
4. **Go live** Testers create registrations in the web app -> within ~2s they appear in every client's dropdowns -> staff print -> documents file to limsDocs and stream into the LIMS Documents page.
5. **Rotate secrets** Immediately run set-super-admin.bat on each client to replace the factory super-admin password.

8 Quick troubleshooting

Symptom	Likely cause & fix
Dropdowns empty	No open registration of that device type — add one (Catalog tab / LIMS). Confirm the printer's device type with status.bat.
'Catalog unavailable' in dialog	Client can't reach the hub AND the share fallback is unreadable. Check the hub is up, port 8000, and the .vcp\catalog files exist.
Enrollment fails (409)	Device name already used — pick a unique one.
Document stuck 'held'	Expected without a valid registration — assign it in the hub's Held tab.
Document 'forward_failed'	Hub can't reach Supabase — check internet + keys in Settings; it retries automatically.
Nothing prints at all	Run fix-queue.bat; read %ProgramData%\VirtualCloudPrinter\log.txt; check failed\.

Verified end-to-end

This pipeline was tested with real components: real print -> Ghostscript PDF -> hub filing -> hold -> assign -> cloud forwarding, plus catalog sync, retry recovery, DPAPI password encryption and the print dialog logic. 29/29 hub checks and 7/7 dialog checks passed on the build machine.

Appendix A LIMS integration endpoints

Three directions. Full request/response examples and how to build your own endpoint are in **documents/LIMS-Integration-Guide.md**; this is the map.

A · FETCH FROM LIMS (hub <- LIMS) — keep the catalog current

POST /rest/v1/rpc/catalog_version -> a change watermark (timestamptz)
GET /rest/v1/registrations?select=reg_no,product,status,device_type,
registration_tests(test_name)&status=eq.open -> open registrations + tests
GET /rest/v1/device_types?select=id -> the device-type list

hub polls every poll_seconds; refetches only when the watermark changes

B · SEND TO LIMS (hub -> LIMS) — forward every filed PDF

POST /storage/v1/object/<bucket>/<path> (Bearer, application/pdf, x-upsert:false; 409=ok)
POST /rest/v1/documents?on_conflict=storage_path (apikey+Bearer,
Prefer: resolution=merge-duplicates) -> idempotent upsert of the row
retry: exponential backoff 5 s -> 300 s, never gives up (status forward_failed -> forwarded)

storage_path fixed at enqueue = the idempotency key (retry never duplicates)

C · QUERY THE HUB (your system -> hub :8000) — read / manage

POST /ingest/<token> (X-Device-Token) file + metadata -> filed | held
GET /catalog?device_type=X (X-Device-Token) the client dropdown data
GET/POST /device-types, POST /admin/devices (X-Enroll-Key) enrollment
GET /api/documents[?status] · /api/documents/{id}/file · POST .../assign (X-Admin-Token)
GET/POST /api/registrations · /api/device-types · /api/settings · /api/devices (X-Admin-Token)

Auth planes

Plane	Header	Grants
Device token	X-Device-Token (or token in the ingest URL)	ingest + read the catalog
Enroll key	X-Enroll-Key	list device types, enroll a device
Admin token	X-Admin-Token	everything under /api/* + the dashboard
(open)	-	GET /healthz and GET / (dashboard HTML) only

Point the hub at your LIMS

```
# hub/data/config.json (or the dashboard Settings tab)
{ "supabase_url": "https://YOUR-LIMS-API", "bucket": "lims-docs", "poll_seconds": 2 }
# key: SUPABASE_SERVICE_KEY env var, or DPAPI-encrypted in config (Windows).
# no supabase_url set -> LOCAL MODE: manage the catalog via /api/* , forward queue idles.
```

Build a custom (non-Supabase) endpoint: implement A (3 routes) and B (2 routes) with the shapes above; the only functions that issue them are `catalog_sync_task()` / `_refresh_from_supabase()` (fetch) and `_forward_one()` (send) in `hub/app.py`.